DIGITAL EGYPT PIONEERS INITIATIVE (DEPI) — ROUND 5

# LUMORA LEARNING PLATFORM

Phase 4: System Analysis & Design — Section 1: System Overview & Software Architecture

| Project Attribute | Specification Details |
|---|---|
| **Project Title** | Lumora Learning Platform (Enterprise E-Learning Ecosystem) |
| **Team Name** | Code Raiders (6 Members) |
| **Program & Track** | DEPI Round 5 — Full-Stack .NET Web Development |
| **Training Partner & Group** | AST — Group Code: CAI5_SWD5_S6 |
| **Architectural Paradigm** | N-Tier Architecture with ASP.NET Core MVC (Razor Views) & Entity Framework Core |
| **Document Deliverable** | Phase 4: System Analysis & Design — Part 1: System Overview & Architecture |

## Team Roster & Role Allocation (Code Raiders)

| # | Member Full Name | DEPI ID | Assigned Domain & Architectural Responsibility |
|---|---|---|---|
| **1** | Mohamed Mostafa Sileem Ibrahim (Team Lead) | 21161803 | Architecture & Integration: N-Tier Solution Setup, DI Container, Design Patterns Coordination, KPI Services & Azure Deployment |
| **2** | Osama Mamdouh Saber Abdelnaeem | 21039424 | Database & Authentication: ERD Modeling, EF Core DbContext, ASP.NET Identity (RBAC), Migrations & Seed Data |
| **3** | Habiba Khaled Fathallah Ebrahim | 21132559 | Course Management & CRUD: Core Course Builder, Sections & Lessons Hierarchy, Course Metadata & Draft/Publish Workflow |
| **4** | Mariem Badwey Abdelsatar Ali | 21171234 | Student Learning Experience: Course Details View, Video.js Player Integration, Progress Tracking, Quizzes & Course Reviews |
| **5** | Ahmed Hany Mohamed Allam | 21060002 | Enrollment, Payment, Azure Blob & Certificates: Checkout, Stripe Test Mode Integration, Azure Blob Video Upload & iTextSharp PDF Certificates |
| **6** | Ahmed Mohammed Abdelfattah Mohammed | 21147537 | Shared UI, Instructor Dashboard & Admin: Bootstrap 5 Master Layout, Home/Catalog UI, Instructor Analytics & Admin Dashboard (Chart.js) |

# 1.1 Problem Statement & Project Objectives

## 1.1.1 Executive Context & Problem Statement

The rapid expansion of online education has exposed critical architectural and operational gaps in existing digital learning platforms. Organizations, independent educators, and students currently face three major structural challenges when utilizing conventional e-learning systems:

- **1. High Complexity in Traditional LMS Platforms:** Traditional academic Learning Management Systems (such as Moodle or Open edX) are heavily geared toward institutional administration, suffering from cluttered user interfaces, steep learning curves, and rigid configuration workflows that alienate independent course creators and self-paced learners.
- **2. Restrictive Commercial Marketplaces:** Commercial marketplaces impose aggressive revenue-sharing commissions, restrict instructor control over student engagement metrics, and lock course content into proprietary ecosystems with limited customization.
- **3. Media Piracy & Unverifiable Credentials:** Standard web-based learning portals frequently lack enterprise-grade digital rights protection—leaving raw video URLs vulnerable to unauthorized scraping and hotlinking—and issue static completion certificates that can be trivially forged without cryptographic or database-backed verification.

To resolve these challenges, the Code Raiders team is engineering the Lumora Learning Platform: a modular, secure, and developer-transparent e-learning ecosystem built on ASP.NET Core MVC and N-Tier Architecture. Lumora unifies frictionless course authoring, Azure-secured video delivery via time-limited Signed URLs, PCI-compliant Stripe payment processing, interactive quiz assessments, and verifiable PDF certification within a single cohesive web application.

## 1.1.2 Strategic Project Objectives & KPIs

Lumora's engineering and business goals are structured around measurable SMART objectives across four core pillars:

| Objective Pillar | Strategic Goal | Technical Mechanism in Lumora | Target Success Metric (KPI) |
|---|---|---|---|
| **1. Architectural Modularity** | Establish a highly maintainable, testable, and decoupled codebase suitable for enterprise scaling. | 3-Layer N-Tier Architecture (Lumora.Web, Lumora.Services, Lumora.Data) using Repository, Unit of Work, and DI. | 100% separation of SQL/EF logic from MVC Controllers; zero tight coupling. |
| **2. Content Security & Media Protection** | Eliminate unauthorized video downloading, scraping, and direct URL sharing. | Store video assets in private Azure Blob Storage containers and serve streams via Azure CDN Signed URLs. | Signed URLs expire dynamically; 0% public anonymous blob access. |
| **3. Seamless Learning & Assessment** | Maximize student engagement and verify comprehension before | Interactive Video.js playback tracking, automated lesson progress calculation, and | Real-time progress persistence; 100% completion + passing quiz |

| | | | |
|---|---|---|---|
| | credentialing. | mandatory section quizzes. | score required for certificate. |
| **4. Verifiable Certification & Commerce** | Provide trustworthy digital credentials and secure financial transactions. | Server-side PDF generation via iTextSharp with unique Verification Codes, paired with Stripe Checkout. | Sub-3-second PDF certificate generation; 100% verifiable certificate IDs via public lookup. |
| **5. Governance & Quality Assurance** | Ensure only high-quality, reviewed educational content reaches the public catalog. | Multi-stage Course Approval Workflow (Draft -> Pending Approval -> Published/Rejected) with Admin KPIs. | Role-segregated workflows enforced by ASP.NET Identity RBAC & Chart.js dashboards. |

# 1.2 Use Case Diagram & Detailed Specifications

## 1.2.1 System Actors Catalog

Lumora enforces strict Role-Based Access Control (RBAC) through ASP.NET Core Identity, governing interactions across four human actors and three external system actors:

| Actor Name | Actor Type | Role & Core Responsibilities within Lumora |
|---|---|---|
| **Guest (Unauthenticated Visitor)** | Primary Actor | Browses the public home page, searches and filters the published course catalog, views course previews/syllabi, verifies certificate codes, and registers for an account. |
| **Student (Learner)** | Primary Actor | Enrolls in free or paid courses via Stripe, streams protected video lessons, tracks lesson completion, takes section quizzes, submits 5-star reviews, and downloads verified PDF certificates. |
| **Instructor (Content Creator)** | Primary Actor | Creates and structures courses (Sections & Lessons), uploads video lectures to Azure Blob Storage, manages pricing/discounts, submits drafts for approval, and monitors course revenue/analytics. |
| **Administrator (Platform Admin)** | Primary Actor | Reviews pending courses (Approve/Reject with feedback), manages user accounts and platform categories, and monitors system-wide financial and engagement KPIs via Chart.js. |
| **Stripe Payment Gateway** | External System Actor | Processes PCI-compliant credit card transactions in Test Mode, validates payment intents, and returns confirmation callbacks to finalize student enrollment. |
| **Azure Blob Storage & CDN** | External System Actor | Stores high-definition instructional videos in private cloud containers and generates time-limited Shared Access Signature (SAS) / Signed URLs for authorized streaming. |
| **iTextSharp PDF Engine** | External Component Actor | Synthesizes dynamic, tamper-evident PDF completion certificates embedded with student metadata, course title, completion date, and a unique cryptographic Verification Code. |

## 1.2.2 System Use Case Diagram

The following diagram illustrates the complete Lumora Learning Platform system boundary, highlighting primary actors on the left, external integrations on the right, and explicit <<include>> and <<extend>> functional dependencies:

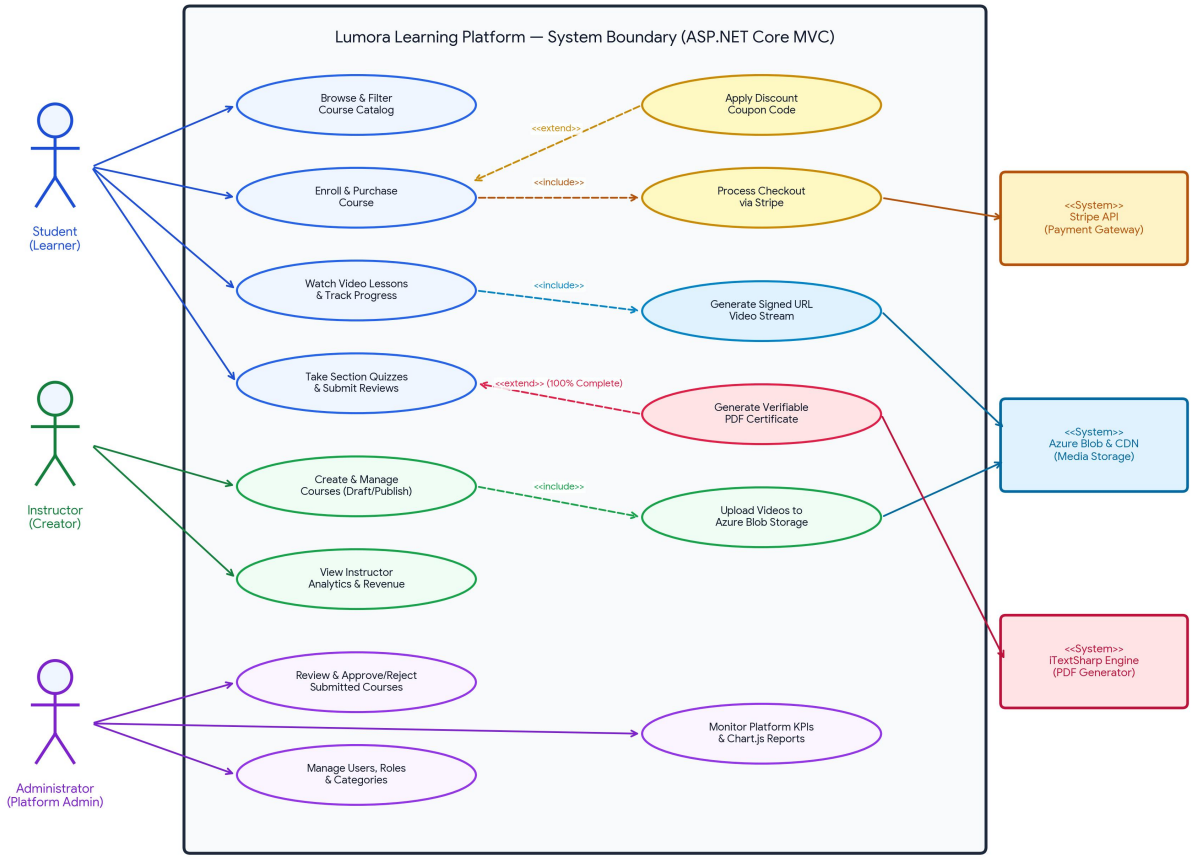



*Figure 1.1: Lumora Learning Platform — Master UML Use Case Diagram*

## 1.2.3 Structured Use Case Descriptions

Below are the detailed engineering specifications for the core end-to-end use cases governing the platform:

### UC-01: Course Creation, Curriculum Structuring & Media Upload

| Specification Field | Detailed Description |
|---|---|
| **Use Case ID & Name** | UC-01: Create & Manage Course (Draft to Submission) |
| **Primary Actor** | Instructor |
| **Secondary Actors** | Azure Blob Storage & CDN |
| **Preconditions** | User is authenticated and authorized with the 'Instructor' role via ASP.NET Identity. |
| **Postconditions** | Course metadata, sections, and lessons are persisted in SQL Server; video files are stored in |

| | |
|---|---|
| | Azure Blob Storage; course status is set to 'Draft' or 'PendingApproval'. |
| **Relationships** | <<include>> Upload Videos to Azure Blob Storage |
| **Main Success Scenario** | 1. Instructor navigates to Instructor Dashboard and selects 'Create New Course'.<br>2. Instructor enters course metadata (Title, Description, Category, Difficulty Level, Base Price, Thumbnail) via Razor form.<br>3. CourseController validates ViewModel and invokes CourseService to save the Course entity in 'Draft' status.<br>4. Instructor adds Sections and Lessons, selecting video files for upload.<br>5. AzureBlobService streams video files to private Azure Blob container and stores the blob URI in the Lesson record.<br>6. Instructor clicks 'Submit for Review', transitioning Course state from 'Draft' to 'PendingApproval'. |
| **Alternate / Exception Flows** | 5a. Invalid File Format/Size: Model validation rejects non-MP4 files or files exceeding size limits and displays a Bootstrap validation alert.<br>5b. Azure Network Timeout: Upload fails gracefully; transaction rolls back via UnitOfWork without corrupting the Lesson record. |

### UC-02: Course Quality Assurance & Admin Approval Workflow

| Specification Field | Detailed Description |
|---|---|
| **Use Case ID & Name** | UC-02: Review & Approve/Reject Submitted Courses |
| **Primary Actor** | Administrator |
| **Secondary Actors** | Observer Notification Service |
| **Preconditions** | Administrator is logged in; at least one course exists with status 'PendingApproval'. |
| **Postconditions** | Course status transitions to 'Published' (visible in public catalog) or 'Rejected' (returned to Instructor with review notes). |
| **Relationships** | Triggers Observer Pattern notification to Instructor upon state transition. |
| **Main Success Scenario** | 1. Admin opens the Admin Dashboard and views the 'Pending Course Approvals' queue.<br>2. Admin inspects course syllabus, lesson structure, and streams sample video lectures.<br>3. Admin verifies compliance with quality standards and clicks 'Approve & Publish'.<br>4. CourseService updates Course status to 'Published', records ApprovalTimestamp, and commits via UnitOfWork.<br>5. Course immediately appears in the public Student Catalog. |
| **Alternate / Exception Flows** | 3a. Course Rejected: Admin selects 'Reject', enters mandatory rejection feedback notes, and submits. Course status reverts to 'Rejected', allowing the Instructor to address issues and resubmit. |

### UC-03: Course Enrollment & Stripe Payment Processing

| Specification Field | Detailed Description |
|---|---|
| **Use Case ID & Name** | UC-03: Enroll & Purchase Course |
| **Primary Actor** | Student |
| **Secondary Actors** | Stripe Payment Gateway API |
| **Preconditions** | Student is authenticated; target course has status 'Published'; Student is not already enrolled. |
| **Postconditions** | Payment transaction is recorded; Enrollment entity is created; Student gains immediate access to course player. |
| **Relationships** | <<include>> Process Checkout via Stripe \| <<extend>> Apply Discount Coupon Code |
| **Main Success Scenario** | 1. Student views Course Details page and clicks 'Enroll Now' (or adds course to Shopping Cart).<br>2. Optional <<extend>>: Student enters a promotional coupon code; PricingStrategy recalculates the net payable amount.<br>3. EnrollmentController invokes StripeService to create a Stripe Checkout Session (Test Mode) and redirects the Student.<br>4. Student submits test card details on Stripe's PCI-compliant checkout page.<br>5. Stripe verifies payment and redirects back to Lumora's confirmation callback action.<br>6. EnrollmentService creates the Payment and Enrollment records atomically via UnitOfWork and redirects Student to the Course Player. |
| **Alternate / Exception Flows** | 4a. Payment Declined / Cancelled: Stripe returns failure status; Lumora logs the failed attempt, preserves the cart, and displays a clear retry message without creating an Enrollment record. |

### UC-04: Secure Video Streaming, Progress Tracking & Certificate Issuance

| Specification Field | Detailed Description |
|---|---|
| **Use Case ID & Name** | UC-04: Watch Lessons, Pass Quizzes & Generate Verifiable PDF Certificate |
| **Primary Actor** | Student |
| **Secondary Actors** | Azure CDN (Signed URLs), iTextSharp PDF Engine |
| **Preconditions** | Student has an active Enrollment in the course. |
| **Postconditions** | LessonProgress and QuizAttempt records are updated; upon 100% completion, a verifiable PDF certificate is generated and stored. |
| **Relationships** | <<include>> Generate Signed URL Video Stream \| <<extend>> Generate Verifiable PDF Certificate (at 100% completion) |
| **Main Success Scenario** | 1. Student opens the Course Player Razor View.<br>2. CourseService requests a time-limited Signed URL from AzureBlobService for the selected lesson video and injects it into Video.js.<br>3. Upon video completion, an AJAX/MVC post updates LessonProgress and recalculates |

| | |
|---|---|
| | overall Enrollment completion percentage.<br>4. Student completes the end-of-section Quiz; QuizService evaluates answers against the passing threshold.<br>5. <<extend>> When overall progress reaches 100% and required quizzes are passed, the Observer Pattern triggers PdfCertificateService.<br>6. iTextSharp generates a custom PDF certificate with a unique Verification Code (e.g., LUM-2026-XXXX), saves metadata to the Certificates table, and unlocks the 'Download Certificate' button. |
| **Alternate / Exception Flows** | 4a. Quiz Score Below Threshold: Student is shown detailed assessment feedback and prompted to retry the quiz; certificate generation remains locked until passed. |

# 1.3 Functional & Non-Functional Requirements

## 1.3.1 Functional Requirements Specification (FR)

The table below enumerates the core functional capabilities across all platform modules, prioritized using the MoSCoW methodology and mapped to the responsible team domain:

| Req ID | Module / Domain | Functional Requirement Description | Priority | Owner |
|---|---|---|---|---|
| **FR-AUTH-01** | Identity & Auth | System shall support user registration, login, logout, and password hashing using ASP.NET Core Identity. | Must Have | Member 2 |
| **FR-AUTH-02** | Identity & Auth | System shall enforce Role-Based Access Control (RBAC) across Student, Instructor, and Admin roles. | Must Have | Member 2 |
| **FR-CRS-01** | Course Management | Instructors shall create, update, and delete courses with metadata (title, price, level, category, thumbnail). | Must Have | Member 3 |
| **FR-CRS-02** | Course Management | Instructors shall organize course curriculum into ordered Sections and individual Lessons. | Must Have | Member 3 |
| **FR-CRS-03** | Course Management | Instructors shall save courses as 'Draft' and submit completed courses for Admin approval. | Must Have | Member 3 |
| **FR-MED-01** | Cloud Media | System shall upload instructor video files directly to private Azure Blob Storage containers. | Must Have | Member 5 |
| **FR-MED-02** | Cloud Media | System shall serve video streams exclusively to enrolled students using time-limited Azure Signed URLs. | Must Have | Member 5 |
| **FR-LRN-01** | Student Learning | Students shall browse and search the published catalog by keyword, category, price, and rating. | Must Have | Member 4 & 6 |
| **FR-LRN-02** | Student Learning | System shall track student lesson completion and dynamically compute course progress percentage. | Must Have | Member 4 |

| | | | | |
|---|---|---|---|---|
| **FR-LRN-03** | Student Learning | Students shall take multiple-choice section quizzes with automated grading and instant feedback. | Must Have | Member 4 |
| **FR-LRN-04** | Student Learning | Enrolled students shall submit a 1-to-5 star rating and written review per course. | Should Have | Member 4 |
| **FR-PAY-01** | Commerce & Payment | System shall provide a shopping cart, discount coupon evaluation, and Stripe Test Mode checkout. | Must Have | Member 5 |
| **FR-CERT-01** | Certification | System shall automatically generate downloadable PDF certificates via iTextSharp upon 100% completion. | Must Have | Member 5 |
| **FR-CERT-02** | Certification | Each certificate shall contain a unique Verification ID verifiable via a public lookup endpoint. | Must Have | Member 5 |
| **FR-ADM-01** | Admin & Analytics | Admins shall approve or reject submitted courses and manage platform categories and user statuses. | Must Have | Member 6 |
| **FR-ADM-02** | Admin & Analytics | Instructor and Admin dashboards shall render interactive KPI charts (revenue, enrollments) via Chart.js. | Must Have | Member 1 & 6 |

### 1.3.2 Non-Functional Requirements Specification (NFR)

To guarantee enterprise-grade reliability, security, and user experience, Lumora adheres to strict quality attributes:

| NFR ID | Quality Category | Requirement & Architectural Constraint | Target Metric / Verification |
|---|---|---|---|
| **NFR-SEC-01** | Security & Auth | All sensitive routes and Controller actions must be protected via [Authorize(Roles=...)] attributes and Anti-Forgery CSRF tokens. | Zero unauthorized endpoint access; 100% POST forms include @Html.AntiForgeryToken(). |
| **NFR-SEC-02** | Media DRM Protection | Raw Azure Blob container access must be set to Private; video streams must use SAS/Signed URLs that expire automatically. | Signed URLs expire within 60-120 minutes; direct anonymous URL requests return HTTP 403. |
| **NFR-SEC-03** | Payment Compliance | Raw credit card numbers must never touch or be stored in the Lumora SQL database; all card handling is delegated to Stripe. | 100% PCI-DSS delegation via Stripe Hosted Checkout / Elements in Test Mode. |
| **NFR-PRF-01** | Performance | Server-side Razor view rendering and EF Core database queries must utilize asynchronous programming (async/await) and .AsNoTracking() for read-only catalog views. | Catalog & dashboard page load times < 1.5 seconds under standard load. |
| **NFR-UIX-01** | Usability & Responsive UI | All Razor views must be built with Bootstrap 5 responsive grid layouts supporting desktop, tablet, and mobile viewports. | 100% mobile responsiveness across Chrome, Edge, Firefox, and Safari. |

| | | | |
|---|---|---|---|
| **NFR-MNT-01** | Maintainability | Strict N-Tier separation of concerns: Controllers cannot reference DbContext directly; all database operations must pass through Services and UnitOfWork. | Enforced via C# project reference hierarchy in Visual Studio Solution. |
| **NFR-AVL-01** | Reliability & Hosting | Application shall be deployed to Microsoft Azure App Services backed by Azure SQL Database with automated connection resiliency. | 99.5% uptime availability during staging and capstone evaluation. |

# 1.4 Software Architecture & Enterprise Design Patterns

## 1.4.1 Architectural Style Justification: N-Tier with ASP.NET Core MVC

The Code Raiders team made a strategic engineering decision to architect Lumora as a multi-layered N-Tier application powered by ASP.NET Core MVC (Razor Views) rather than a decoupled SPA/Web API. This architectural choice provides four major advantages aligned with enterprise .NET standards:

- **1. Cohesive Full-Stack Velocity:** Server-side Razor rendering eliminates cross-origin resource sharing (CORS) complexities, duplicate client/server state synchronization, and dual-pipeline deployment overhead, allowing the 6-member team to deliver rich features rapidly.
- **2. Hardened Cookie-Based RBAC Security:** ASP.NET Core Identity integrates natively with encrypted HTTP-only authentication cookies, role claims, and Razor Tag Helpers, providing stronger out-of-the-box protection against XSS token theft compared to localStorage JWTs.
- **3. Strict Separation of Concerns (N-Tier):** By splitting the Visual Studio solution into dedicated class libraries (Presentation, Business Logic, Data Access, Core Domain), business rules remain completely independent of UI frameworks and database engines.
- **4. Unified Cloud Deployment:** The entire platform compiles into a single deployable artifact hosted on Azure App Service, seamlessly connecting to Azure SQL Database, Azure Blob Storage, and Stripe.

## 1.4.2 High-Level N-Tier Architecture Diagram

Figure 1.2 illustrates Lumora's physical and logical N-Tier layering, detailing the internal components of each layer and its integration boundaries with external cloud services:

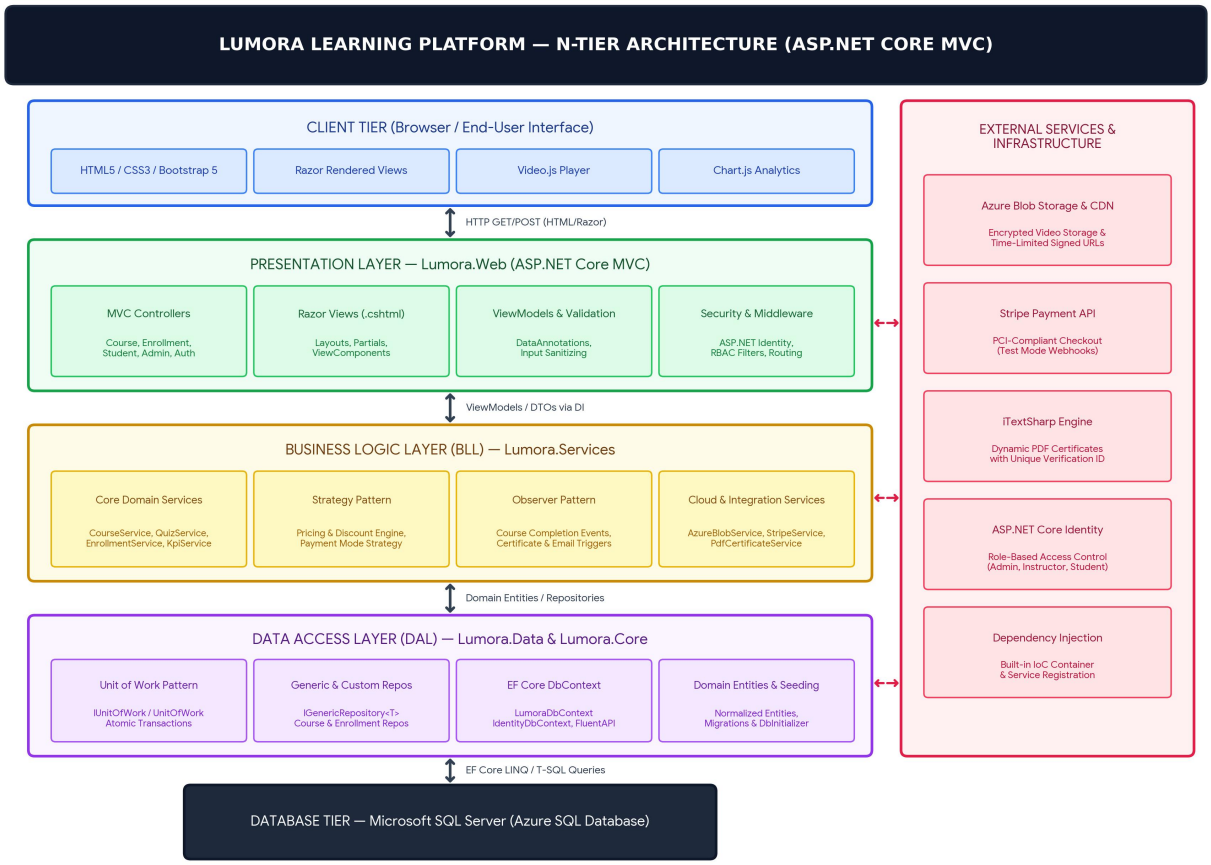



*Figure 1.2: Lumora Learning Platform — Multi-Layered N-Tier Software Architecture*

## 1.4.3 End-to-End Request Lifecycle & Layer Responsibilities

Every user interaction in Lumora follows a deterministic, unidirectional dependency pipeline from the browser down to the database and back through the Razor View Engine:

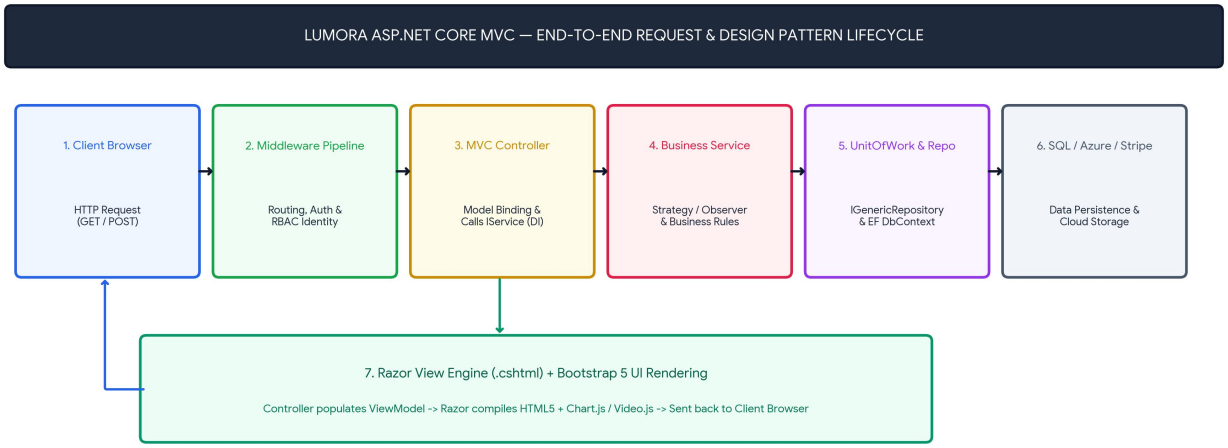



*Figure 1.3: End-to-End HTTP Request Execution & Design Pattern Lifecycle in Lumora MVC*

| Architectural Layer | Project Assembly | Core Components | Dependency Rules & |
|---|---|---|---|

| | | | Responsibilities |
|---|---|---|---|
| **1. Presentation Layer** | Lumora.Web (ASP.NET Core MVC) | Controllers, Razor Views (.cshtml), ViewModels, ViewComponents, wwwroot (Bootstrap 5, Chart.js, Video.js), Identity Middleware. | Handles HTTP requests, model validation, and UI rendering. References Lumora.Services and Lumora.Core. Strictly forbidden from referencing Lumora.Data directly. |
| **2. Business Logic Layer (BLL)** | Lumora.Services (.NET Class Library) | Domain Services (CourseService, EnrollmentService, QuizService, KpiService), Cloud Adapters (AzureBlobService, StripeService, PdfCertificateService), Strategy & Observer Implementations. | Enforces domain rules, calculates progress/KPIs, orchestrates external APIs, and manages transactions via IUnitOfWork. References Lumora.Data and Lumora.Core. |
| **3. Data Access Layer (DAL)** | Lumora.Data (.NET Class Library) | LumoraDbContext (inherits IdentityDbContext), GenericRepository<T>, Specific Repositories, UnitOfWork, EF Core Fluent API Configurations, Migrations, DbInitializer (Seed Data). | Encapsulates all EF Core LINQ queries and SQL Server persistence. References Lumora.Core. |
| **4. Core Domain Layer** | Lumora.Core (.NET Class Library) | Domain Entities (ApplicationUser, Course, Section, Lesson, Enrollment, Quiz, Review, Certificate, Payment), Enums (CourseStatus, DifficultyLevel), Repository & Service Interfaces. | Zero external project dependencies. Acts as the shared contract layer across the entire solution. |

## 1.4.4 Enterprise Design Patterns Implemented

To ensure clean code, high testability, and adherence to SOLID principles, Lumora integrates five foundational software design patterns across its N-Tier architecture:

| Design Pattern | Category | Concrete Implementation in Lumora | Architectural Benefit |
|---|---|---|---|
| **1. Repository Pattern** | Data Access / Structural | IGenericRepository<T> provides standard CRUD operations (GetByIdAsync, GetAllAsync, AddAsync, Update, Delete), extended by specialized repositories (e.g., ICourseRepository for eager-loading Sections, Lessons, and Reviews). | Abstracts Entity Framework Core queries away from Business Services, enabling unit testing with mocked repositories and preventing duplicate LINQ queries. |
| **2. Unit of Work Pattern** | Data Access / Behavioral | IUnitOfWork coordinates multiple repositories sharing a single LumoraDbContext instance and exposes CompleteAsync() to commit changes in a single atomic database transaction. | Guarantees data integrity during multi-table operations (e.g., creating a Payment record, inserting an Enrollment, and updating Course enrollment |

| | | | |
|---|---|---|---|
| | | | count simultaneously). |
| **3. Dependency Injection (IoC)** | Architectural / Creational | All Services, Repositories, UnitOfWork, and external clients (Stripe, Azure Blob) are registered via interface contracts in Program.cs using Scoped lifetime management. | Eliminates hardcoded object instantiation ('new' keyword), enforces Dependency Inversion (SOLID), and simplifies component replacement. |
| **4. Strategy Pattern** | Behavioral | IPricingStrategy & IPaymentStrategy interfaces allow runtime switching between standard pricing, percentage coupon discounts, seasonal promotions, and Mock Checkout vs. Stripe Test Mode checkout. | Follows the Open/Closed Principle: new discount rules or payment gateways can be added without modifying existing Checkout or Enrollment service code. |
| **5. Observer Pattern** | Behavioral / Event-Driven | When a Student completes the final lesson or passes the final quiz (CourseCompletedEvent), subscribed observers (CertificateGenerationObserver, NotificationObserver) execute automatically. | Decouples progress tracking from side-effects: LessonProgressService does not need to know how PDF certificates or instructor notifications are generated. |

## 1.4.5 Visual Studio Solution Structure (Lumora.sln)

Below is the standardized project and directory tree configured by Member 1 (Architecture Lead) inside the GitHub repository (lumora-learning-platform) to enable conflict-free parallel development across all 6 team members:

```
Lumora.sln
├── src/
│   ├── Lumora.Core/                        # Shared Domain Entities, Enums & Contracts
│   │   ├── Entities/                       # ApplicationUser, Course, Section, Lesson, Enrollment,
Quiz, Review, Certificate
│   │   ├── Enums/                          # CourseStatus (Draft, Pending, Published, Rejected),
CourseLevel
│   │   └── Interfaces/                     # IGenericRepository<T>, IUnitOfWork, ICourseService,
IPaymentStrategy
│   │
│   ├── Lumora.Data/                        # Data Access Layer (EF Core & SQL Server)
│   │   ├── Context/                        # LumoraDbContext.cs (IdentityDbContext<ApplicationUser>)
│   │   ├── Configurations/                 # Fluent API EntityTypeConfigurations (Keys, Indexes,
Cascade Rules)
│   │   ├── Migrations/                     # EF Core Schema Migrations
│   │   ├── Repositories/                   # GenericRepository.cs, CourseRepository.cs,
UnitOfWork.cs
│   │   └── Seeding/                        # DbInitializer.cs (Default Roles, Admin User & Sample
Courses)
│   │
│   ├── Lumora.Services/                    # Business Logic Layer & External Integrations
│   │   ├── Implementations/                # CourseService, EnrollmentService, QuizService,
KpiAnalyticsService
│   │   ├── Cloud & External/               # AzureBlobStorageService, StripePaymentService,
PdfCertificateService
```

```
│   │   ├── Patterns/
│   │   │   ├── Strategies/                 # StandardPricingStrategy, CouponDiscountStrategy,
StripeCheckoutStrategy
│   │   │   └── Observers/                  # CourseCompletionSubject, CertificateIssuerObserver
│   │   └── DTOs/                           # Data Transfer Objects & Service Results
│   │
│   └── Lumora.Web/                         # Presentation Layer (ASP.NET Core MVC)
│       ├── Controllers/                    # HomeController, AccountController, CourseController,
StudentController,
│       │                                   # EnrollmentController, InstructorController,
AdminController
│       ├── Views/                          # Razor Views (.cshtml) organized by Controller + Shared
(_Layout, Partials)
│       ├── ViewModels/                     # Strongly-typed UI ViewModels with DataAnnotations
│       ├── wwwroot/                        # Static Assets (Bootstrap 5, Custom CSS, Chart.js,
Video.js)
│       ├── appsettings.json                # ConnectionStrings, AzureBlobSettings, StripeSettings
│       └── Program.cs                      # Middleware Pipeline, Identity Config & DI Container
Registration
└── tests/
    └── Lumora.UnitTests/                   # xUnit Test Suite for Services, Strategies & KPI
Calculations
```